

Sistemas Operativos

Trabalho Prático

Serviço de Indexação e Pesquisa de Documentos

Grupo 19

Elementos do grupo:

Nome: José Pedro Flores Novais

Número: a108395

Nome: Samir Mansour

Número: a105856

Nome: Fábio Mendes Castelhana

Número: a105728

Sumário

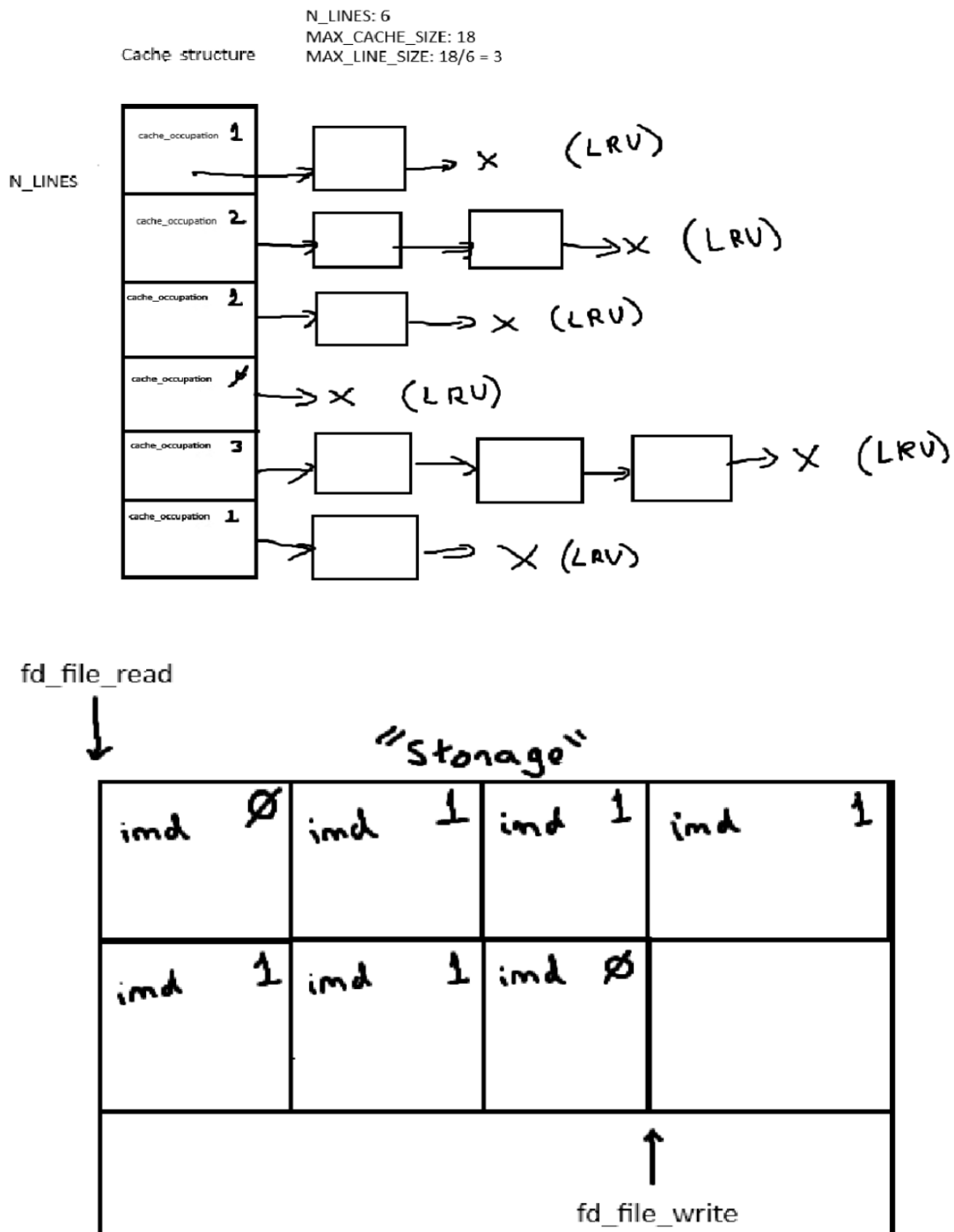
1. Introdução
2. Arquitetura do Sistema
3. Comunicação entre Processos
4. Funcionalidades implementadas
 - 4.1. Indexação de documentos (-a)
 - 4.2. Consulta de meta-informação (-c)
 - 4.3. Remoção de documentos (-d)
 - 4.4. Pesquisa em documento (-l)
 - 4.5. Pesquisa global (-s)
 - 4.6. Paragem do servidor (-f)
5. Funcionalidades avançadas e Otimizações
 - 5.1. Pesquisa concorrente
 - 5.2. Persistência
 - 5.3. Cache de meta-informação

1. Introdução

No âmbito da Unidade Curricular de Sistemas Operativos da Universidade do Minho do 2º ano de Licenciatura em Ciências da Computação, o grupo vem, por meio deste relatório, explicar as principais decisões tomadas, no que diz respeito à arquitetura de processos, funcionalidades, e da escolha e uso concreto dos mecanismos de comunicação.

2. Arquitetura do Sistema

Estão representados abaixo a cache e o storage, de maneira a tornar mais claro as suas estruturas:



3. Comunicação entre Processos

Dentre os processos presentes no projeto, há diferentes formas de comunicação entre eles.

Entre o cliente e o servidor, o meio de comunicação que foi utilizado foi o FIFO, em que o cliente cria um FIFO fixo chamado PIPE_TO_SERVER, e o cliente sempre abre esse FIFO em modo escrita e envia uma struct FileInfo. Para que o cliente possa receber as informações do servidor, o cliente cria seu próprio FIFO temporário, com base no PID (os passos dessa criação temporária não foram mencionadas nas explicações das funcionalidades implementadas, a fim

de que pudéssemos dar mais ênfase a outras informações mais relevantes). O servidor abre esse FIFO com base no PID recebido na struct FileInfo, e envia a resposta ao cliente. O cliente lê sua resposta e deleta esse FIFO temporário.

Já entre os processos do próprio servidor, o meio de comunicação que foi utilizado foi o pipe. No comando -s, o objetivo foi que os processos filhos criados com fork() comuniquem ao processo pai quais documentos contêm a palavra procurada. No comando -l, os pipes foram utilizados para criarem uma pipe-line entre processos, neste caso, entre os comandos grep e wc, em que ambos atuam como filhos, e o pai recebe o resultado numérico no final dessa pipe-line.

4. Funcionalidades Implementadas

Nos subtópicos abaixo, serão dadas as principais estratégias e decisões tomadas para cada uma das funcionalidades implementadas.

4.1. Indexação de Documentos (-a)

No cliente, foi utilizada a struct FileInfo para armazenar as informações que foram escritas na linha de comandos. Depois, essas informações são escritas no FIFO PIPE_TO_SERVER.

O servidor lê essas informações através do PIPE_TO_SERVER, e as copia para outra struct, chamada file_struct, e essas informações são finalmente indexadas no storage. Após isso, uma mensagem é criada com o id atribuído, e ela é escrita no FIFO fifo_client.

Finalmente, o cliente lê a mensagem que foi escrita no fifo_client, e a escreve na tela.

4.2. Consulta de Meta-informação (-c)

No cliente, foi utilizada a struct FileInfo para armazenar as informações que foram escritas na linha de comando (nesse caso, somente o identificador para consultar a meta-informação de um documento). Depois, essas informações são escritas no FIFO PIPE_TO_SERVER.

O servidor lê essas informações através do PIPE_TO_SERVER, e começa a procura pelo documento cujo identificador foi o recebido. Primeiramente, a procura é feita na cache, uma vez que é o mecanismo para acesso mais rápido. Se não for encontrado na cache, a busca é feita no storage e, se for encontrado, o servidor o adiciona à cache para futuras consultas rápidas.

Finalmente, o servidor prepara a mensagem (se não encontrou nada, envia a mensagem dizendo isso), e a escreve no FIFO fd_client. Dessa forma, o cliente lê a mensagem que foi escrita no fd_client, e escreve o seu conteúdo na tela.

4.3. Remoção de documentos (-d)

No cliente, foi utilizada a struct FileInfo para armazenar as informações que foram escritas na linha de comando (nesse caso, somente o identificador da meta-informação a ser removida). Depois, essas informações são escritas no FIFO PIPE_TO_SERVER.

O servidor lê essas informações através do PIPE_TO_SERVER, e começa a procura pelo documento cujo identificador foi o recebido. Primeiramente, a procura é feita na cache, uma vez que é o mecanismo para acesso mais rápido. Se for encontrado na cache, ele é removido. Porém, independentemente se ele é encontrado na cache ou não, o servidor procura o

documento no storage e, se for encontrado, altera o valor do campo `active` da `file_struct` para 0. Isso faz com que o conteúdo real do documento não seja apagado, apenas deixa de estar indexado.

Finalmente, se o documento com o referido identificador foi encontrado e removido, uma mensagem de confirmação é escrita no FIFO `fd_client`. Se não foi encontrado, uma mensagem de negação é escrita no FIFO `fd_client`. Em qualquer um dos casos, o cliente lê a mensagem que foi escrita no `fd_client`, e escreve o seu conteúdo na tela.

4.4. Pesquisa em documento (-l)

No cliente, foi utilizada a struct `Fileinfo` para armazenar as informações que foram escritas na linha de comando. Depois essas informações são escritas no FIFO `PIPE_TO_SERVER`.

O servidor lê essas informações através do `PIPE_TO_SERVER`, e a função `search_keyword` é chamada. Se o documento estiver na cache, a função `grep_wc` é chamada com o caminho do documento. Se não, se ele for encontrado no storage, a função `grep_wc` é chamada com `file_struct.path`, e depois o documento é adicionado à cache.

De maneira simplificada, a função `grep_wc` cria dois pipes: o `pipe1`, que vai de `grep` para `wc`, e o `pipe2`, que vai de `wc` para o processo pai. O processo filho 1 executa o comando `grep` e redireciona a saída para o `pipe1`, e o processo filho 2 recebe como entrada o conteúdo de `pipe1` e redireciona a saída para o `pipe2`. Finalmente, o processo pai lê do `pipe2`, extrai o número de linhas, e grava a mensagem que, posteriormente, é escrita no FIFO do cliente, é lida pelo cliente no seu FIFO. Por fim, a mensagem é escrita no Standart output.

4.5. Pesquisa global (-s)

No cliente, foi utilizada a struct `Fileinfo` para armazenar a informação que foi escrita na linha de comandos. Depois, essa informação é escrita no FIFO `PIPE_TO_SERVER`.

O servidor lê essa informação através do `PIPE_TO_SERVER`, e a função `search_contains_word` é chamada. Primeiramente, ele remove todas as ocorrências de ficheiros desativados em modo a facilitar a divisão de tarefas entre múltiplos processos. O objetivo dela é dividir a pesquisa entre múltiplos processos (isso será melhor explicado no subtópico 5.1), cada processo varre uma fatia do armazenamento, cada processo escreve seus resultados (que no caso, são os ids dos documentos que contêm uma determinada palavra), num pipe, e o processo pai junta tudo em uma mensagem bem formatada.

Finalmente, o servidor escreve a mensagem no FIFO do cliente, e o cliente lê a mensagem pelo seu FIFO, e a mensagem é escrita no Standart output.

4.6. Paragem do servidor (-f)

No cliente, foi utilizada a struct `Fileinfo` para armazenar a informação que foi escrita na linha de comandos (nesse caso, apenas o comando `-f`). Depois, essa informação é escrita no FIFO `PIPE_TO_SERVER`. Logo após isso, o cliente fecha o `fd_to_server`, remove seu FIFO e sai do programa, uma vez que, nesse comando em específico, o cliente não aguarda resposta.

O servidor recebe a struct com a informação (comando `-f`), escreve uma mensagem, e inicia o encerramento. Primeiramente, ele compacta o armazenamento, isto é, se houver arquivos

removidos via comando `-d`, o servidor limpa os registros marcados como inativos (isso será melhor explicado no subtópico 5.2). Logo depois, ele libera os elementos da cache, fecha os arquivos e remove o FIFO, e por fim finaliza o processo, imprimindo uma mensagem de êxito e utilizando o `return 0`.

5. Funcionalidades avançadas e Otimizações

A fim de tornar o projeto mais completo e eficiente, mostraremos as estratégias pensadas e implementadas pelo grupo nos subtópicos abaixo.

5.1. Pesquisa Concorrente

No comando `-s`, a concorrência foi implementada com uma série de etapas: Há a criação dos pipes, sendo que cada processo filho terá um pipe de escrita para comunicar ao pai, depois há uma divisão de trabalho, para que cada processo fique responsável por um número quase fixo de documentos, e depois há a criação dos processos filhos, através do comando `fork()`.

Cada filho abre o arquivo de armazenamento, usa `lseek` para se posicionar no `start_point`, lê o número previamente calculado de documentos e, para cada documento, chama a função `file_contains_word`, que é implementada com a ajuda do `execvp` com o comando `grep`, tendo como retorno a variável `status`, utilizada com a chamada ao sistema `wait()` e `WEXITSTATUS()`. Essa função (`grep -q`) dirá se a palavra existe ou não no documento.

Finalmente, o processo pai fecha todos os pipes de escrita, e lê os resultados parciais de cada filho, e junta todos os ids recebidos em uma única mensagem bem formatada, e aguarda todos os filhos com `wait(NULL)`.

5.2. Persistência

A implementação que utilizou-se, para que fosse possível guardar uma cópia persistente da meta-informação dos documentos, contou com a ajuda da flag `active`, presente na struct `indexed_file`.

Quando o cliente usa o comando `-f`, o que acontece é que, o servidor cria um novo arquivo temporário, chamado “temp”, e copia somente os documentos que estão em disco e que têm o campo `active` igual a 1, e substitui o original, através do comando `rename(“temp”, storage_path)`. Isso faz com que os arquivos que foram desativados através do comando `-d` não sejam persistidos em disco, trazendo uma compactação importante ao disco.

Dessa forma, quando o servidor é iniciado novamente, ele consegue ler todos os registros salvos em disco.

5.3. Cache de meta-informação

A priori, o grupo havia pensado em utilizar, para a cache de meta-informação, uma lista ligada como estrutura de dados, em que cada nodo seria uma struct com as informações necessárias. Todavia, após algum tempo, percebeu-se que uma simples lista ligada não seria a melhor opção, uma vez que ou a remoção ou a inserção de um nodo específico não seria muito eficiente, pois, a depender de como essa estrutura fosse implementada, teríamos que percorrer toda a lista para efetuar uma daquelas operações citadas anteriormente (remoção ou inserção).

A fim de minimizar o problema citado anteriormente, a estrutura de dados utilizada foi uma adaptação de uma tabela de hash com closed addressing, que consiste de um array, de tamanho fixo, de listas ligadas. A sua função de hash é: $\text{hash} = \text{id} \% \text{N_LINES}$, onde o id é o identificador da meta-informação de cada documento, e o N_LINES é o número de linhas da tabela de hash

A política de substituição que foi implementada foi a Least Recently Used (LRU) por linha. Na prática, ela foi implementada com a ajuda da função `remove_least_used`, que é chamada sempre que uma linha da cache está prestes a ultrapassar seu tamanho máximo. A função percorre a lista ligada presente na linha de cache e, para cada elemento, calcula a subtração ($\text{virtual_time} - \text{curr} \rightarrow \text{last_acessed}$), e remove o que teve acesso mais antigo.

Observação: Cada vez que um documento é consultado ou usado para busca, o campo `last_acessed`, da struct `LinkedList`, é atualizado com o `virtual_time` atual. Isso permite que a LRU funcione corretamente, pois há uma linha do tempo artificial para determinar qual item está mais obsoleto.